DATA STRUCTURES ASSIGNMENT

SPLAY TREE

A Detailed Study of Self-Adjusting Binary Search Trees

TEAM

Roll No.	Name
04	Dev Anjirwala
23	Ved Chauhan
25	Preet Dalal
48	Parth Jariwala

INDEX

Sr. No.	Topic / Contents	Page No.
1	Introduction to Trees	3
2	Binary Tree and Basic Terminology	4
3	Binary Search Tree (BST)	5
4	Problem with an Ordinary BST / Skewed BST	5–6
5	What is a Splay Tree?	7
6	Splay Tree Properties and Conceptual View	8
7	Splay Operation and Terminology	9
8	Rotations: The Building Blocks	10
9	Six Splay Tree Rotation Cases	11
10	Splay Operation Flowchart	12
11	Zig Operation	13–14
12	Zag Operation	15–16
13	Zig-Zig Operation	17–18
14	Zag-Zag Operation	19–20
15	Zig-Zag Operation	21–22
16	Zag-Zig Operation	23–24
17	Insertion in a Splay Tree	25–33
18	Searching in a Splay Tree	34–35
19	Deletion in a Splay Tree	36–39
20	Algorithm: Insertion	40
21	Algorithm: Searching	41
22	Algorithm: Deletion	42
23	Time Complexity of Splay Tree	43–45
24	Advantages, Disadvantages and Applications	45–46
25	Comparison of BST, Splay Tree and AVL Tree	46
26	Conclusion	47

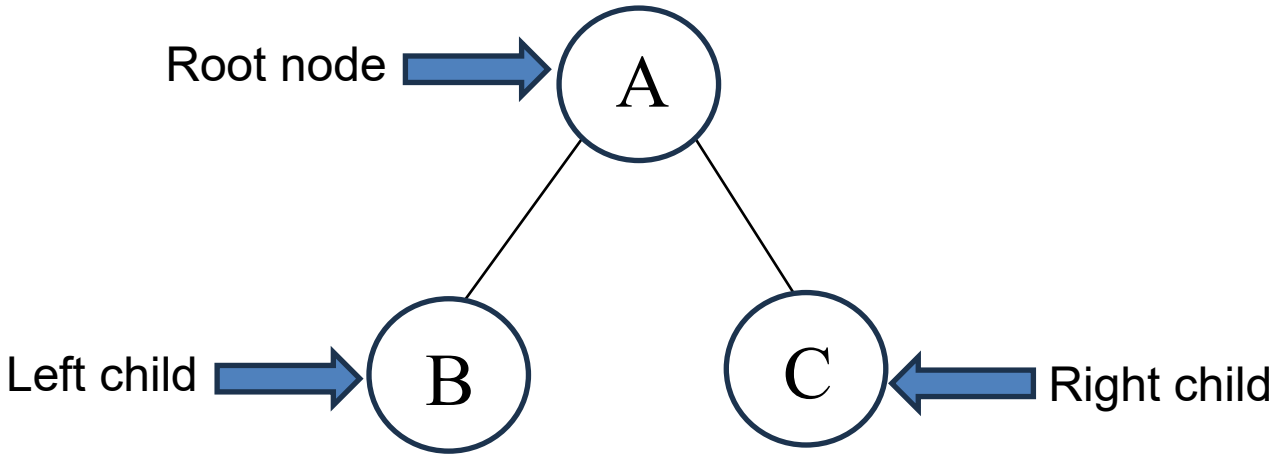
Introduction to Trees

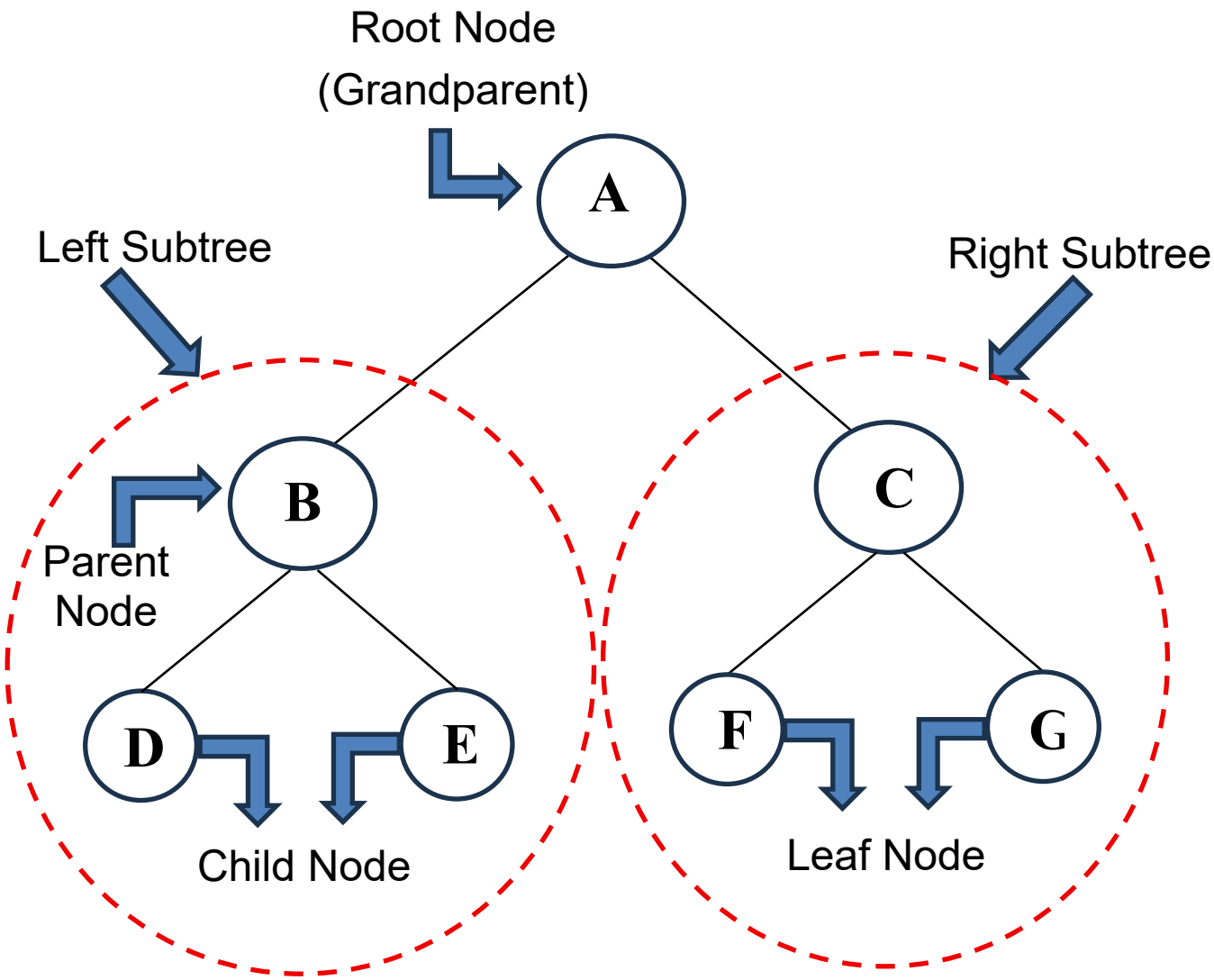
A tree is a non-linear data structure used to represent hierarchical relationships. Unlike arrays and linked lists, tree structures do not store elements in a simple sequential order. A tree consists of nodes connected by edges, with one node generally designated as the root.

Basic Terminology

Term	Meaning
Root	The topmost node of a tree.
Parent	A node that has one or more child nodes.
Child	A node directly connected below a parent.
Leaf	A node with no children.
Edge	The connection between two nodes.
Height	Number of edges on the longest downward path from a node to a leaf.
Subtree	A tree formed by a node and its descendants.

Visual Representation of Terminology





Binary Tree

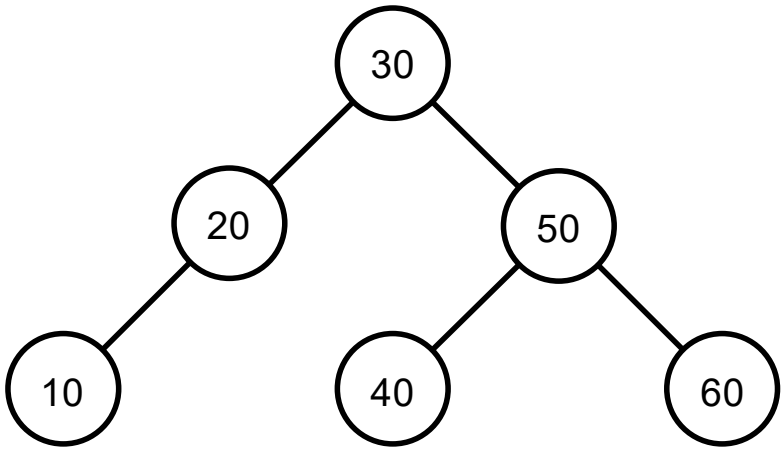
A binary tree is a tree in which each node has at most two children, commonly called the left child and right child. Binary trees form the basis of several important data structures, including Binary Search Trees, AVL Trees, Red-Black Trees and Splay Trees.

Why Trees Are Useful

- They naturally represent hierarchical information.
- They can support efficient searching and organization of ordered data.
- They provide the foundation for many indexing and searching techniques.
- Different balancing strategies can be used to control the height of a tree.

Binary Search Tree

A Binary Search Tree (BST) is a binary tree that maintains an ordering property. For every node, values in the left subtree are smaller than the node's key and values in the right subtree are greater than the node's key, assuming distinct keys.



Binary Search Tree is:

- Binary Tree
- Value is inserted as left child if it’s smaller than the current node's key.
- Value is inserted as right child if it’s greater than the current node's key.

For the above tree, searching for 10 starts at 30, moves to the left because $10 < 30$, and then moves to the left because $10 < 20$.

Problem with an Ordinary BST

The performance of a BST depends strongly on its shape. If values are inserted in an unfavorable order, the tree can become highly skewed.

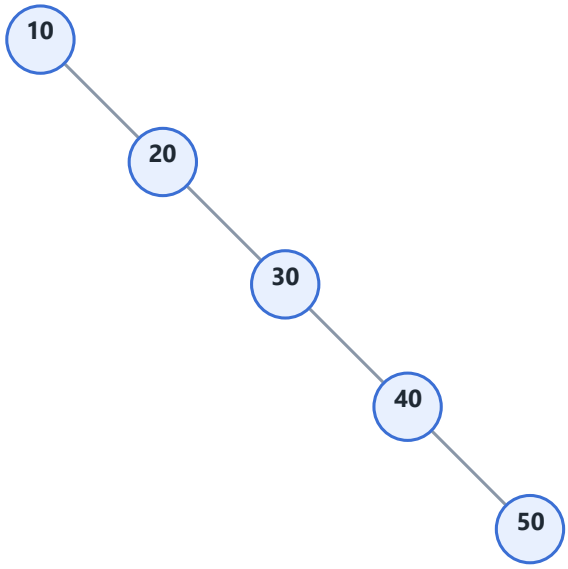
Skewed BST

- A **skewed BST** is a Binary Search Tree in which nodes are arranged mostly in **one direction**, either to the left or to the right. As a result, the tree becomes **unbalanced** and starts to resemble a linked list.

Splay Trees address this problem by reorganizing the tree after access operations, bringing the accessed node closer to the root.

The elements **10, 20, 30, 40 and 50** are inserted in ascending order. Since each new element is greater than the previous one, every node is placed on the **right side** of its parent.

This creates a **right-skewed BST**, where the tree behaves like a linked list. Such a structure can make searching inefficient, which is why a **Splay Tree** performs rotations to bring the recently accessed node closer to the root.



METHOD TO CHECK BST IS CORRECT OR NOT

- **In-Order (Ascending Order)**

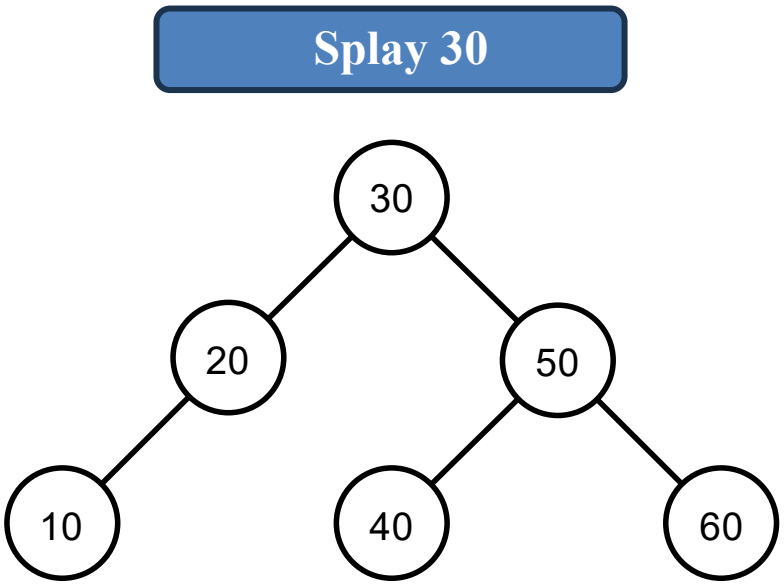
  10 – 20 – 30 – 40 – 50

What is a Splay Tree?

A Splay Tree is a self-adjusting Binary Search Tree in which the node most recently accessed is moved to the root by a sequence of tree rotations called splaying.

A Splay Tree performs a **splay operation** after an access or insertion. The accessed or newly inserted node is moved toward the root by performing suitable rotations. This helps keep frequently accessed nodes closer to the root.

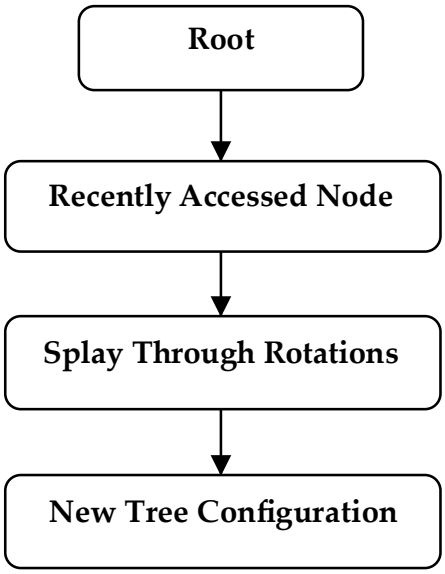
1. **Introduction:**
   Splay Trees were introduced by **Daniel Dominic Sleator and Robert Endre Tarjan in 1985**. They are designed to provide a simple and efficient method for managing dynamic data.
2. **Basic Idea:**
   The main idea of a Splay Tree is to move the most recently accessed or inserted node to the root. This is achieved through a sequence of rotations called **splaying**, which changes the structure of the tree without violating the BST property. A splay operation moves the accessed or newly inserted node toward the root using suitable rotations.
3. **Self-Adjusting Nature:**
   The tree automatically adjusts its structure according to the access pattern. If a particular node is accessed frequently, it tends to remain near the root, making subsequent accesses to that node faster.
4. **Advantages:**
   Splay Trees do not require extra balance information such as a balance factor or height field. Their self-adjusting nature makes them useful for applications where some elements are accessed more frequently than others.
5. **Limitation:**
   A Splay Tree does not always remain balanced. Therefore, a single operation can take **O(n)** time in the worst case when the tree becomes highly unbalanced. However, the tree can reorganize itself through subsequent operations.



Splay Tree Properties

1. The Binary Search Tree ordering property is always preserved.
2. Every access can be followed by a splay operation.
3. The accessed node is moved to the root through rotations.
4. No separate balance field is required in each node.
5. The shape changes dynamically according to the access sequence.
6. The tree can become temporarily unbalanced.
7. Individual operations can take O(n) time (worst case).

Conceptual View



This self-adjustment is what differentiates a Splay Tree from a conventional unbalanced BST.

Splay Operation

The splay tree operations such as insertion, deletion, and search are followed by one special operation called splaying. It is nothing but a set of rotations performed to bring the recently accessed node as the root.

- **Insertion:** To insert a new element into the tree, start by performing a regular binary search tree insertion. Then, apply rotations to bring the newly inserted element to the root of the tree.
- **Deletion**: To delete an element from the tree, first locate it using a binary search tree search. Then, if the element has no children, simply remove it. If it has one child, promote that child to its position in the tree. If it has two children, find the successor of the element (the smallest element in its right subtree), swap its key with the element to be deleted, and delete the successor instead.
- **Search**: To search for an element in the tree, start by performing a binary search tree search. If the element is found, apply rotations to bring it to the root of the tree.
- **Rotation**: The rotations used in a splay tree are either a Zig or a Zig-Zig rotation. A Zig rotation is used to bring a node to the root, while a Zig-Zig rotation is used to balance the tree after multiple accesses to elements in the same subtree.

Splaying is the process of moving a selected node **X** to the root using rotations. The exact rotations depend on the relationship between **X**, its parent and its grandparent.

Based on the heuristic:

- If **X** is accessed once, it is likely to be accessed again.
- After node **X** is accessed, perform "splaying" operations to bring **X** up to the root of the tree.
- Do this in a way that leaves the tree more or less balanced as a whole.

Splay Tree Terminology

• Let **X** be a non-root node, i.e., has at least 1 ancestor.

• Let P be its parent node.

• Let G be its grandparent node (if it exists)

• Consider a path from G to **X** :

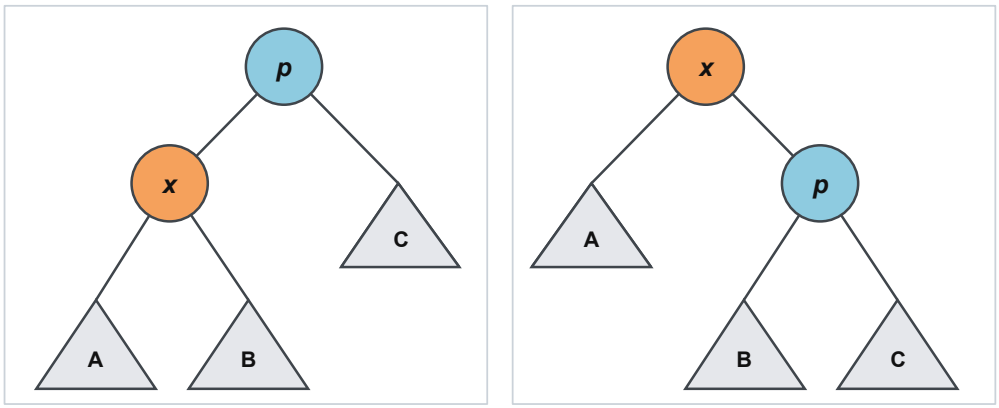
– Each time we go left, we say that we "**zig**"

– Each time we go right, we say that we "**zag**"

Rotations: The Building Block

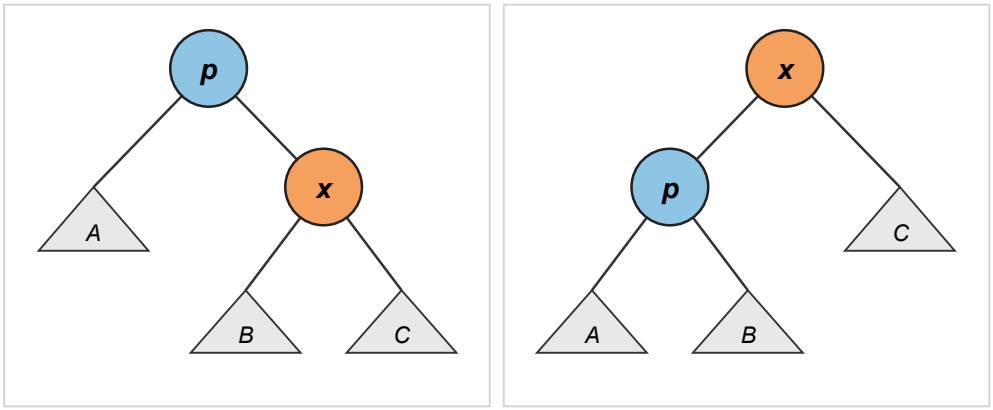
Right rotation (used by Zig)

If x is the **left** child of p, a right rotation makes p the right child of x. The right sub-tree of x (B) changes owner: it was to the right of x and left of p (keys between x and p), so it becomes the **left** child of p.



Left rotation (used by Zag)

If x is the **right** child of p, a left rotation makes p the left child of x. Here the left sub-tree of x (B) is handed to p as its **right** child.



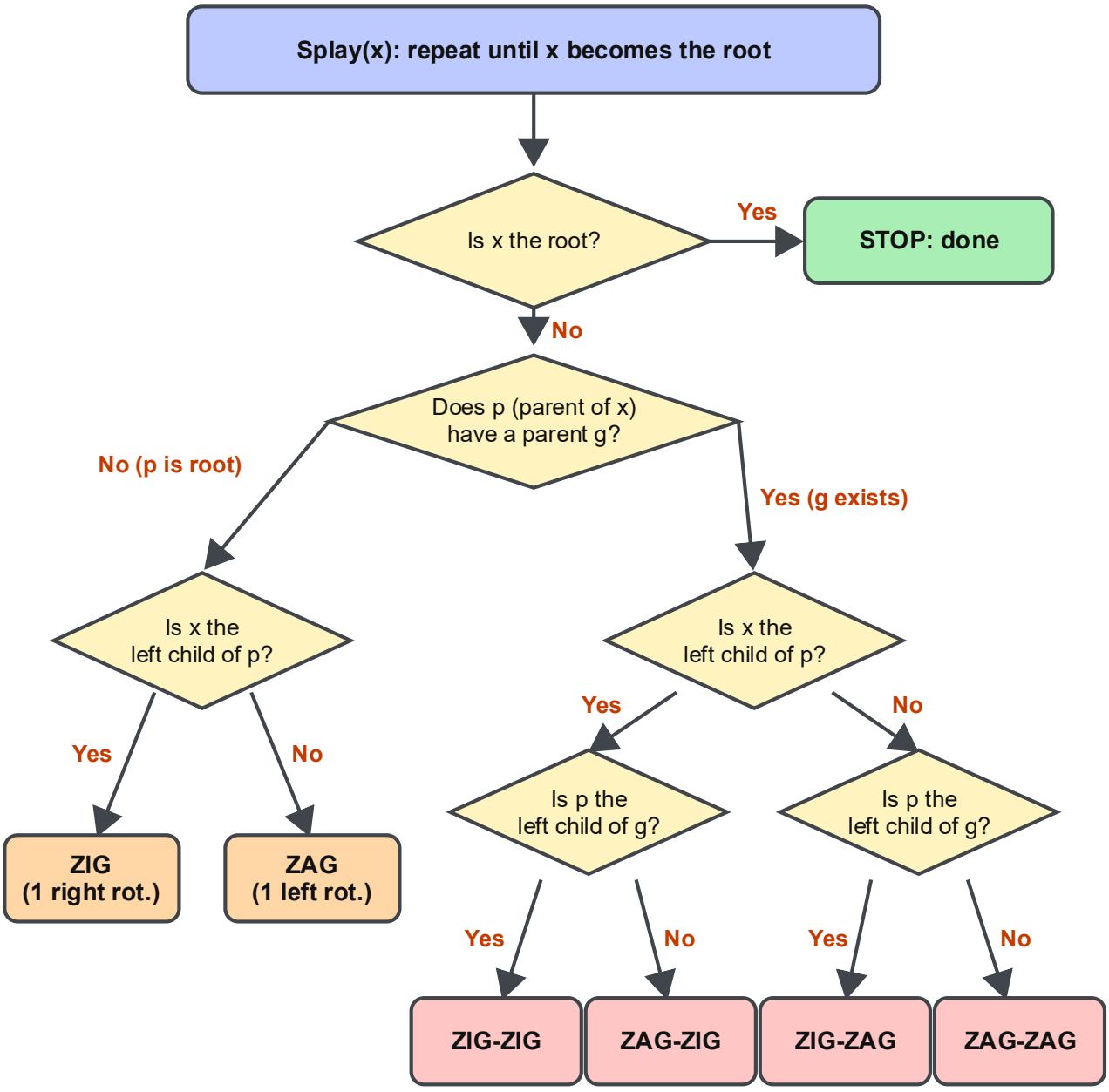
There are 6 possible cases

Case	Condition	Main Action
Zig	**X** is left child of root	One right rotation between X and parent.
Zag	**X** is right child of root	One left rotation between X and parent.
Zag-Zag	**X** and parent are both right children	Two rotations: LEFT ROTATION between **X** and parent LEFT ROTATION between **X** and grandparent
Zig-Zig	**X** and parent are both left children	Two rotations: RIGHT ROTATION between **G** and **P** RIGHT ROTATION between **X** and **P**
Zig-Zag	**X** is right child of parent which is left child of grandparent	Two rotations: RIGHT ROTATION between **X** and **P** LEFT ROTATION between **X** and **G**
Zag-Zig	**X** is left child of parent which is right child of grandparent	Two rotations: RIGHT ROTATION between **X** and parent LEFT ROTATION between **X** and grandparent

The operation is repeated until **X** becomes the root. The rotations preserve the in-order ordering of the Binary Search Tree.

Goal and procedure

Splaying a node x means moving x to the root by a sequence of steps. At each step we look only at the nodes x, p (its parent) and g (its grandparent) and choose exactly one of the six cases. The process repeats until x is the root.

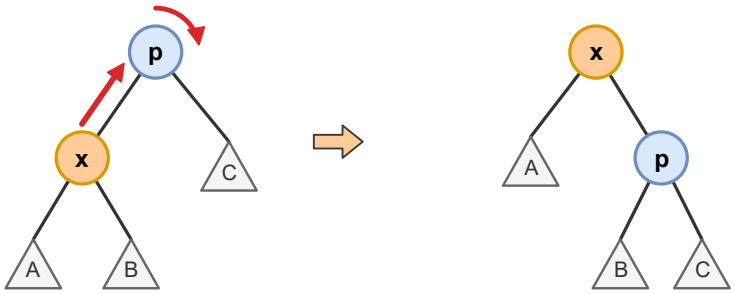


Zig Case

When does this case occur?

The **Zig** case is the simplest of the six. It applies when the node x to be splayed is the **left child of the root**: x has a parent p, and p has no parent (p is the root). Because there is no grandparent, a single rotation is enough to finish the splay.

Zig

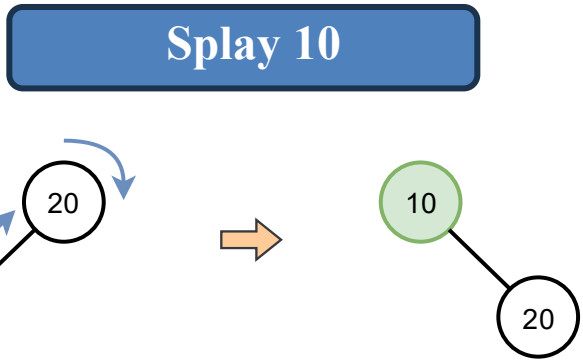


Before
Rotate right at p

After

Example 1:

The following diagrams illustrate the searching of the data 10.



Perform the Zig rotation (Right Rotation) to make the target node (10) as root.

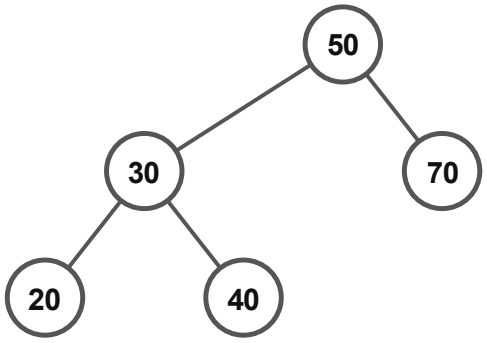
Steps

1. Identify the selected node x.
2. Check that its parent is the root.
3. If x is the left child, perform a right rotation.

4. The selected node becomes the root.

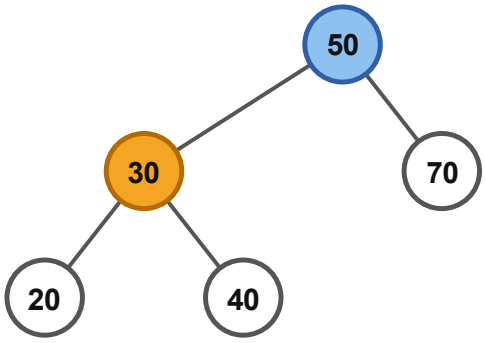
Example 2:

The following diagrams illustrate the searching of the data 30.



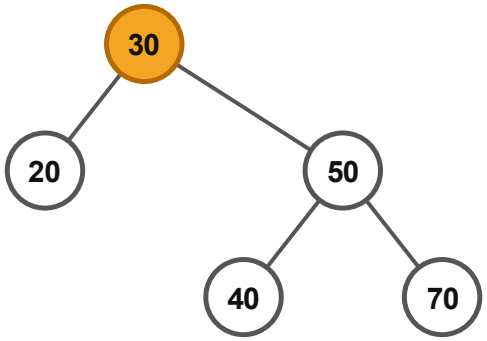
(a) Initial tree

The splay tree before the operation.



(b) Search path

Search path: 50 -> 30. Key 30 is FOUND ; x, p and g are marked.



(c) Step 1: Zig

Zig: 30 is the left child of the root 50. Right rotation of 30 over 50; 30 becomes the root.

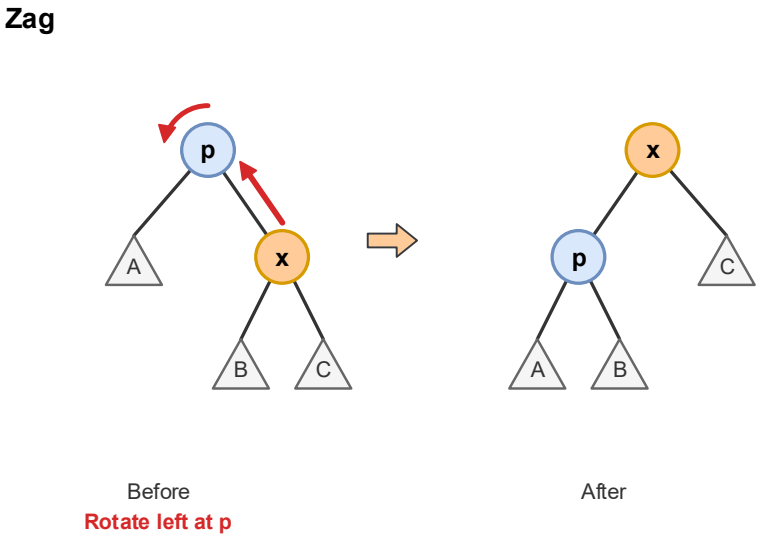
Key points to remember

- Zig needs **no grandparent**: p is the root.
- It is a single **right rotation** of x over p.
- Sub-tree B (the right child of x) is transferred to p as its **left** child.

Zag Case

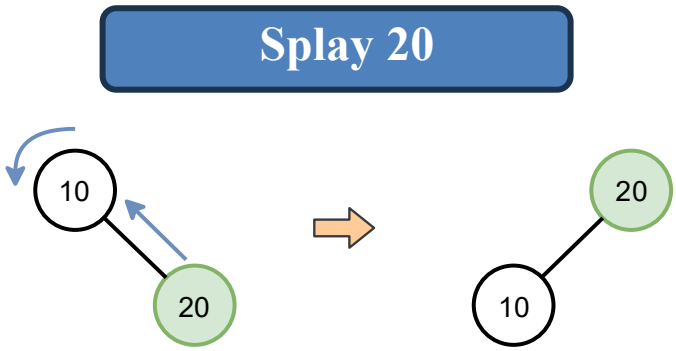
When does this case occur?

It applies when x is the **right child of the root**: its parent p is the root, and x lies on the right of p. A single **left rotation** finishes the splay.



Example 1:

The following diagrams illustrate the searching of the data 20.



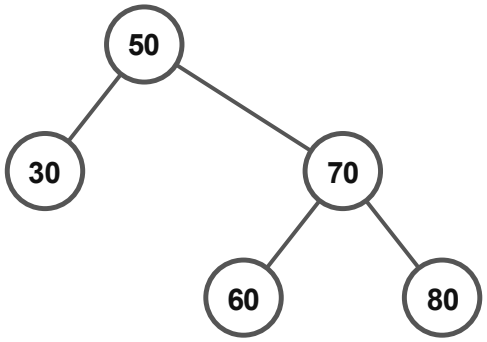
Perform the Zag rotation (Left Rotation) to make the target node (20) as root.

Steps

1. Identify the selected node x.
2. Check that its parent is the root.
3. If x is the right child, perform a left rotation.
4. The selected node becomes the root.

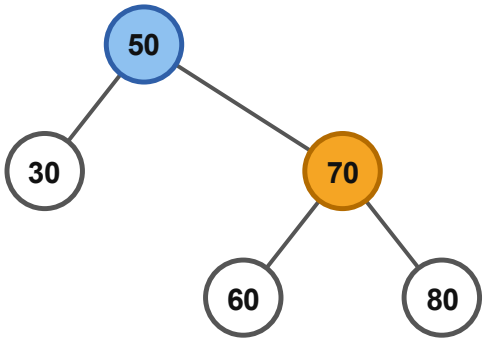
Example 2:

The following diagrams illustrate the searching of the data 70.



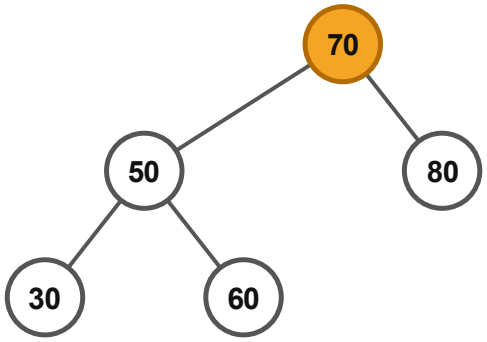
(a) Initial tree

The splay tree before the operation.



(b) Search path

Search path: 50 -> 70. Key 70 is FOUND; x, p and g are marked.



(c) Step 1: Zag

Zag: 70 is the right child of the root 50. Left rotation of 70 over 50; 70 becomes the root.

Key points to remember

- Zag needs **no grandparent**: p is the root.
- It is a single **left rotation** of x over p.
- Sub-tree B (the left child of x) becomes the **right** child of p.

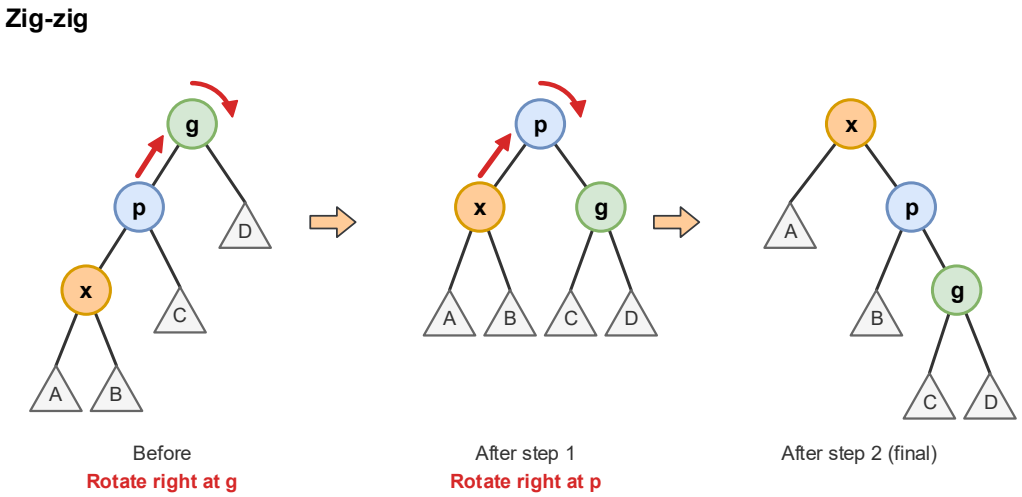
Zig-Zig Case

When does this case occur?

The **Zig-Zig** case occurs when the selected node x and its parent p are both on the **left** side of the grandparent g.

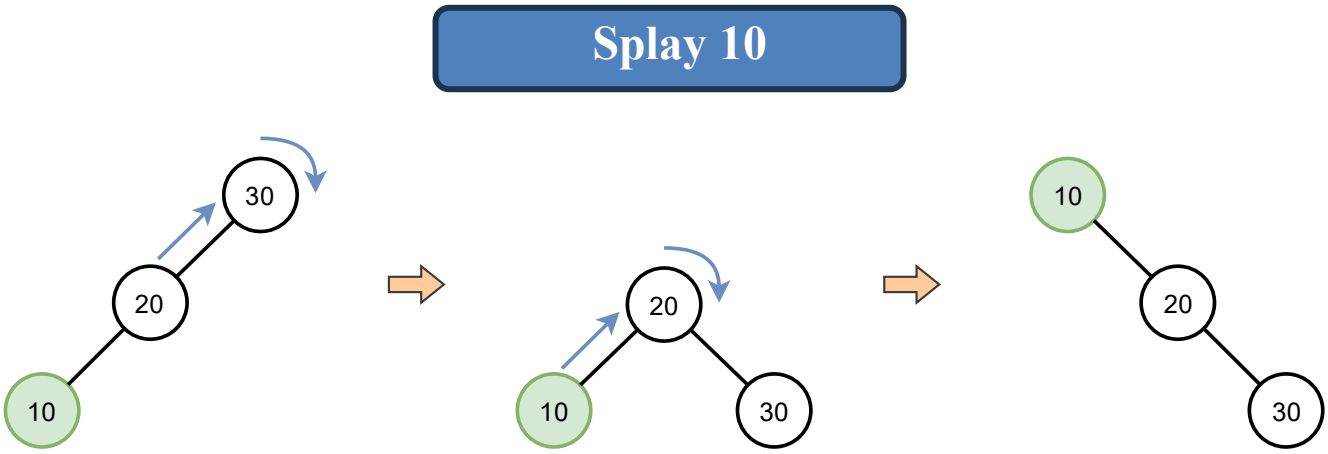
The **Zig-Zig** case applies when x has both a parent p and a grandparent g, **and** x and p are both **left children**: x is the left child of p, and p is the left child of g. The three nodes form a straight line leaning to the left.

Zig-Zig rotates the **parent p over g first**, and only then rotates x over p.



Example 1:

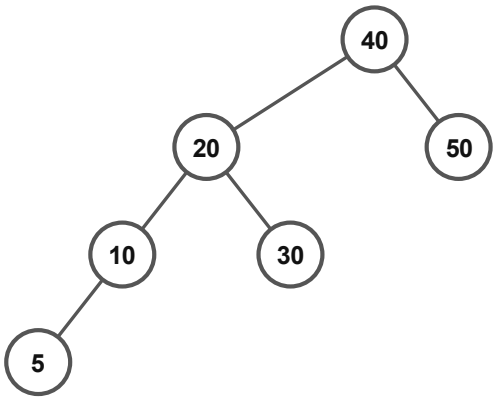
The following diagrams illustrate the searching of the data 10.



Perform the Zig-Zig rotation(right rotation b/w g and p followed by right rotation b/w x and p) to make the target node (10) as root. Note that the grandparent (30) is rotated first followed by the parent (20).

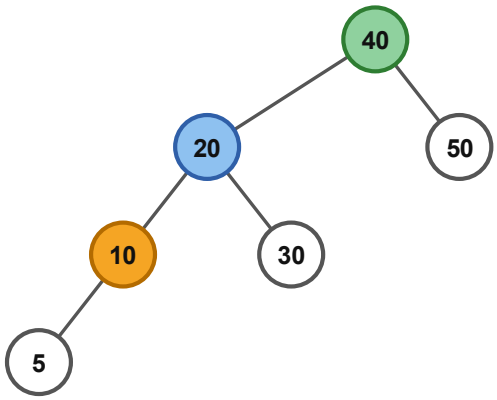
Example 2:

The following diagrams illustrate the searching of the data 10.



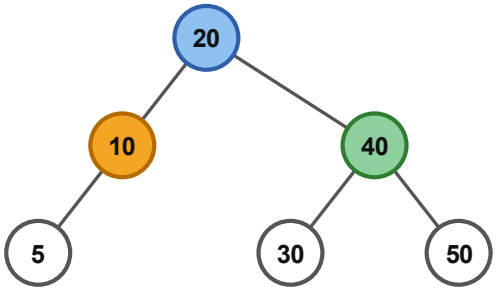
(a) Initial tree

The splay tree before the operation.



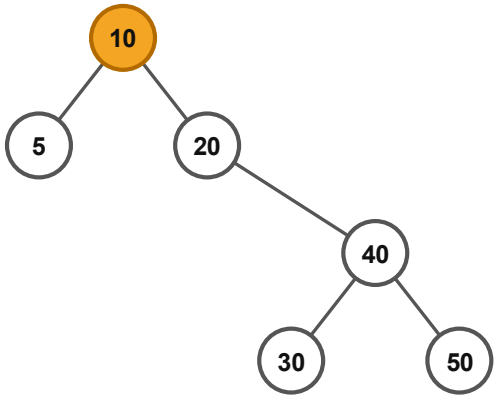
(b) Search path

Search path: 40 -> 20 -> 10. Key 10 is FOUND ; x, p and g are marked.



(c) Step 1: Zig-Zig, rotation 1

Zig-Zig: 10, 20, 40 lies on a straight left line. Rotation 1 (parent first): Right rotation of 20 over 40.



(d) Step 1: Zig-Zig, rotation 2

Rotation 2: Right rotation of 10 over 20.

Important Point

For Zig-Zig Case, the **rotation is performed on the grandparent first** followed by the parent. This is required to maintain the optimal tree balance and to keep the recently accessed items closer to the root node.

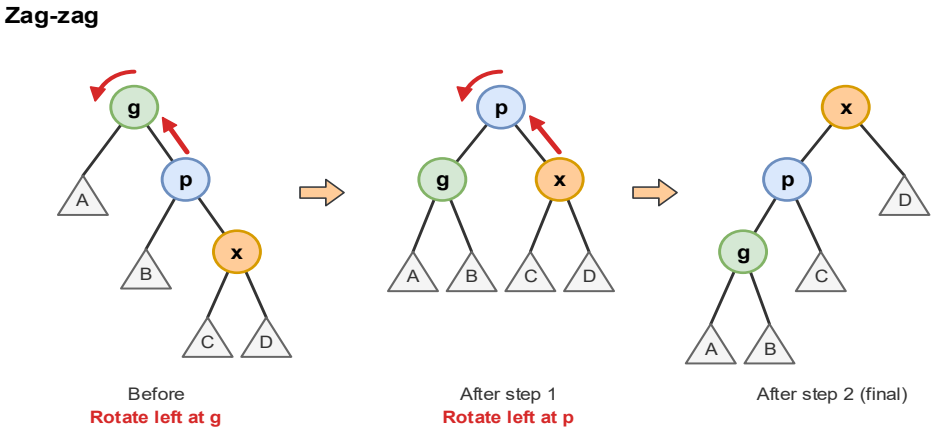
Zag-Zag Case

When does this case occur?

The **Zag-Zag** case occurs when the selected node x and its parent p are both on the **right** side of the grandparent g.

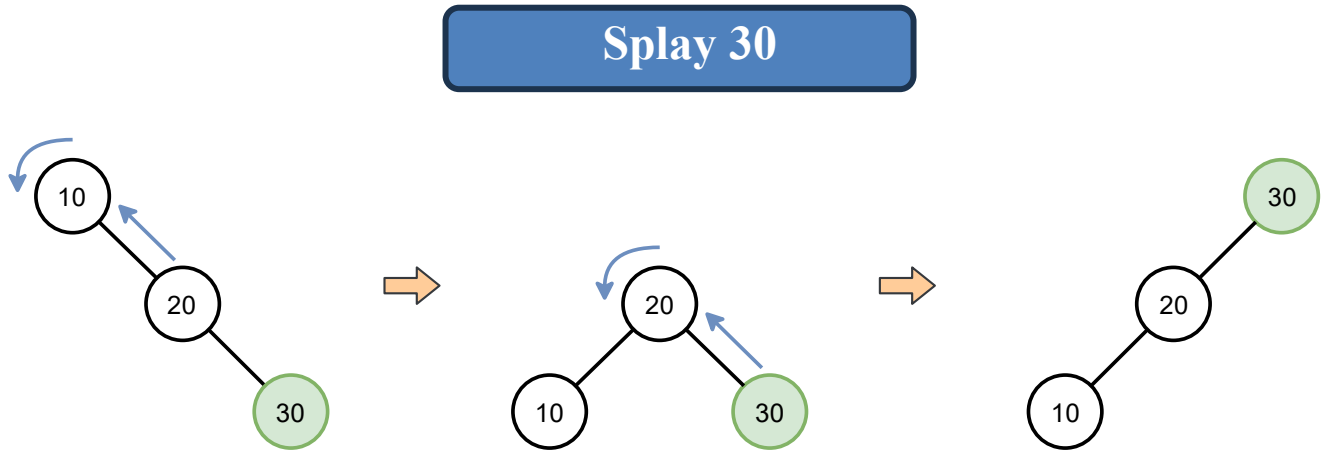
The **Zag-Zag** case is the mirror image of Zig-Zig. It applies when x has a parent p and grandparent g, **and** both x and p are **right children**: x is the right child of p and p is the right child of g. The three nodes form a straight line leaning to the right.

Again the parent is rotated first: p is rotated over g with a **left rotation**, and then x is rotated over p with a second **left rotation**.



Example 1:

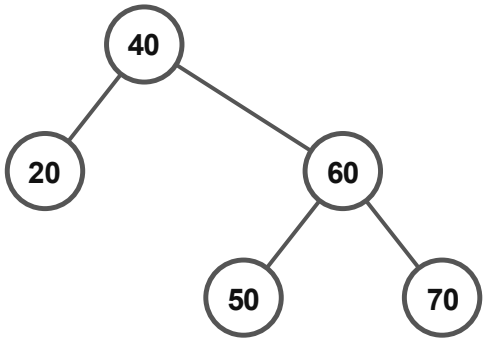
The following diagrams illustrate the searching of the data 30.



Perform the Zag-Zag rotation(left rotation b/w g and p followed by left rotation b/w x and p) to make the target node (30) as root. Note that the grandparent (10) is rotated first followed by the parent (20).

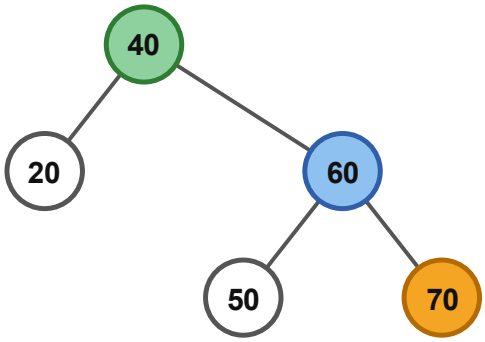
Example 2:

The following diagrams illustrate the searching of the data 70.



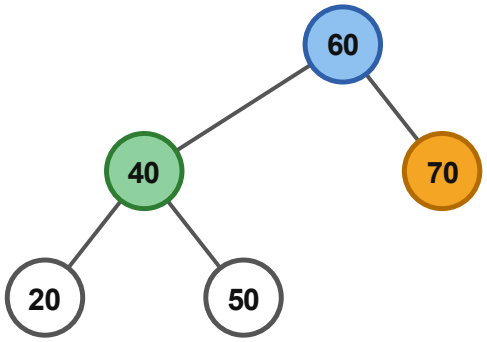
(a) Initial tree

The splay tree before the operation.



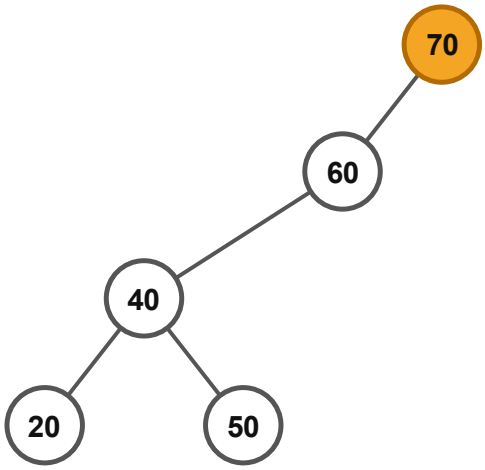
(b) Search path

Search path: 40 -> 60 -> 70. Key 70 is FOUND ; x, p and g are marked.



(c) Step 1: Zag-Zag, rotation 1

Zag-Zag: 70, 60, 40 lies on a straight right line. Rotation 1 (parent first): Left rotation of 60 over 40.



(d) Step 1: Zag-Zag, rotation 2

Rotation 2: Left rotation of 70 over 60.

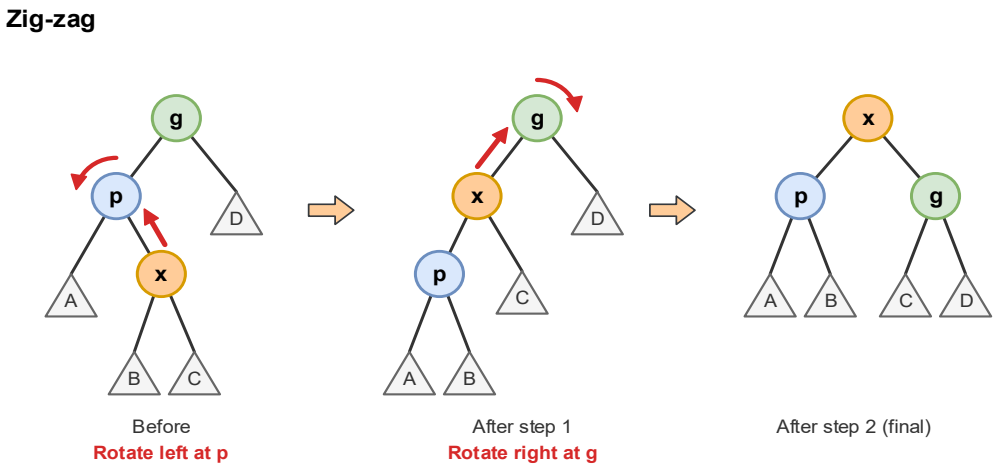
Important Point

For Zag-Zag Case, the **rotation is performed on the grandparent first** followed by the parent. This is required to maintain the optimal tree balance and to keep the recently accessed items closer to the root node.

Zig-Zag Case

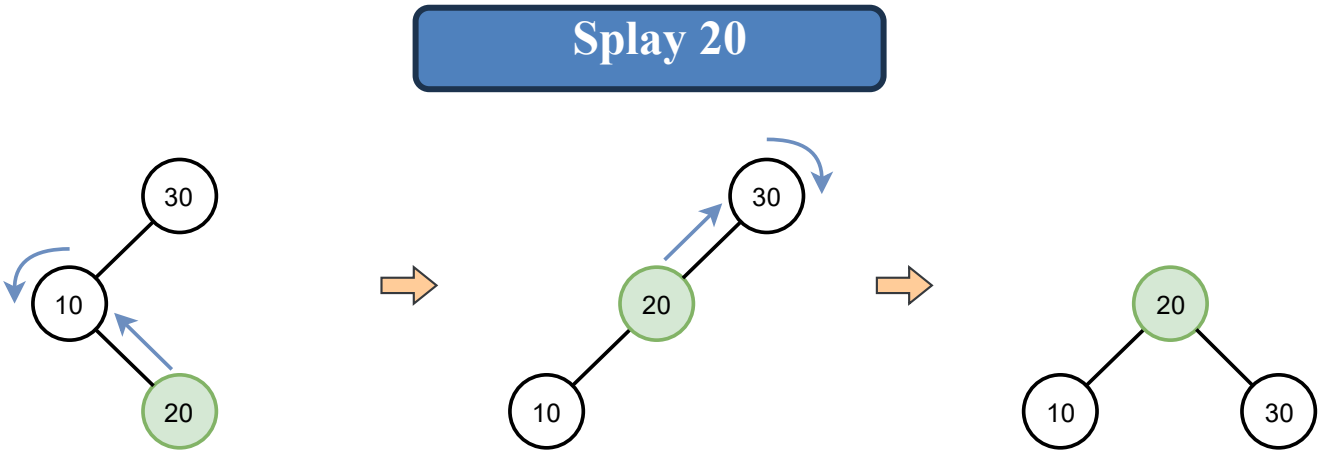
The **Zig-Zag** case applies when x has a parent p and a grandparent g, and the path **bends**: p is the **left** child of g, but x is the **right** child of p. The path g → p → x goes left then right, forming a **"<"** shape.

Here the parent cannot be rotated first without making matters worse; instead, x itself is rotated **twice**: **first a left rotation over p (so x takes p's place), then a right rotation over g.**



Example 1:

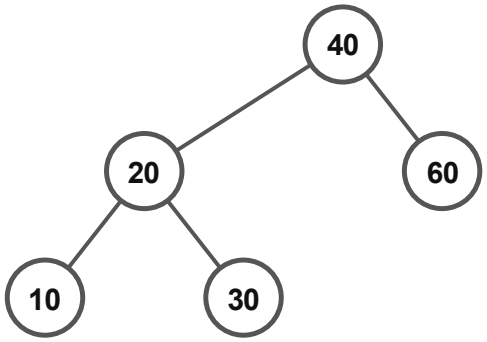
The following diagrams illustrate the searching of the data 20.



Perform the Zig-Zag rotation (first a left rotation over p **(so x takes p's place)**, then a right rotation over g) to make the target node (20) as root. Note that the parent node (10) is rotated first followed by the grandparent (30).

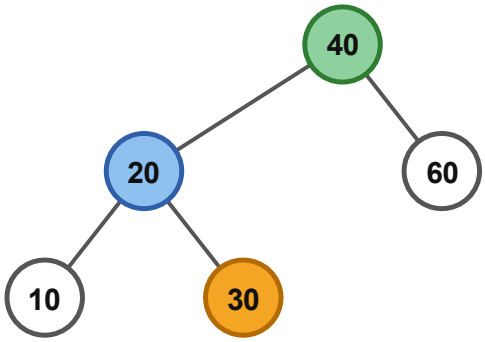
Example 2:

The following diagrams illustrate the searching of the data 30.



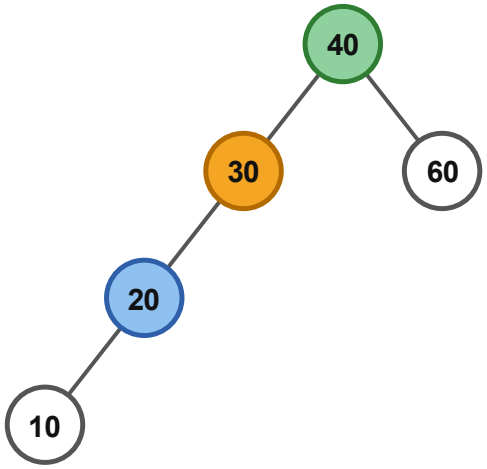
(a) Initial tree

The splay tree before the operation.



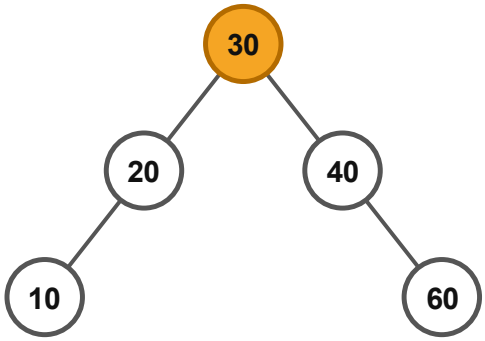
(b) Search path

Search path: 40 -> 20 -> 30. Key 30 is FOUND ; x, p and g are marked.



(c) Step 1: Zig-Zag, rotation 1

Zig-Zag: the path 40 -> 20 -> 30 bends. Rotation 1: Left rotation of 30 over 20.



(d) Step 1: Zig-Zag, rotation 2

Rotation 2: Right rotation of 30 over 40.

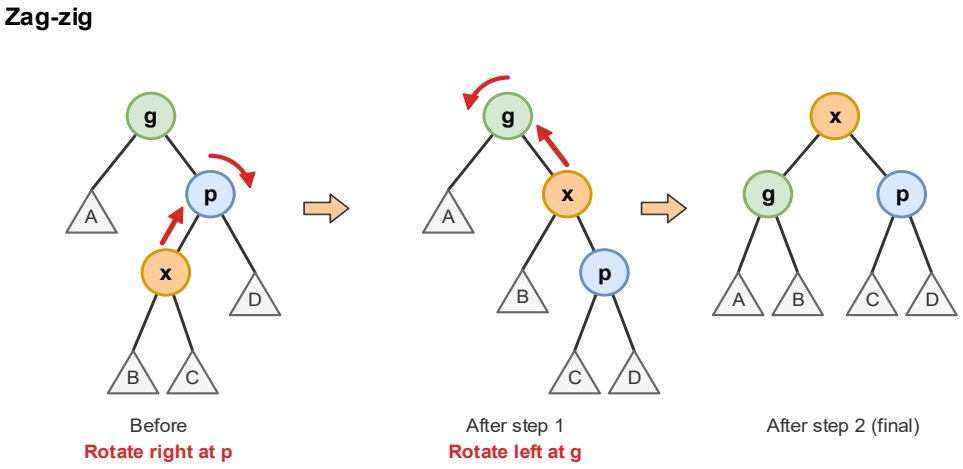
Important Point

For Zig-Zag Case, the rotation is performed on the parent first followed by the grandparent. The reverse order does not work in this case, because performing the first rotation on the grandparent would affect the original position of the splaying node.

Zag-Zig Case

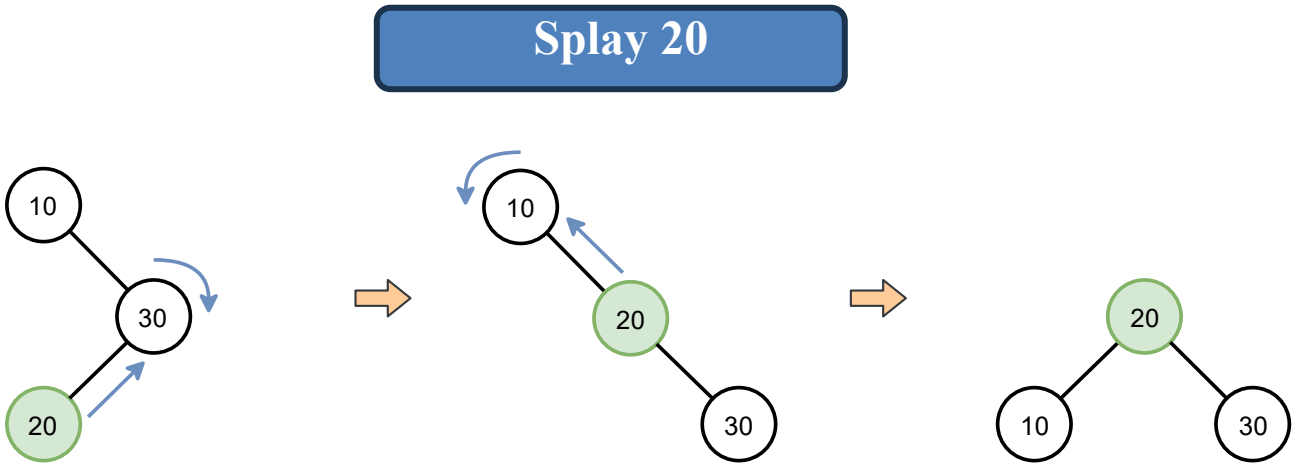
The **Zag-Zig** case is the mirror image of Zig-Zag. It applies when p is the **right** child of g but x is the **left** child of p. The path g → p → x goes right then left, forming a **">"** shape.

As in Zig-Zag, x is rotated twice: a **right rotation** over p, followed by a **left rotation** over g.



Example 1:

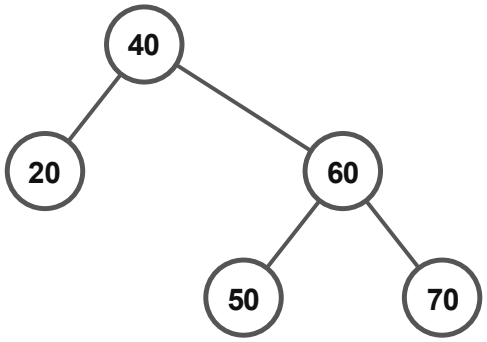
The following diagrams illustrate the searching of the data 20.



Perform the Zag-Zig rotation (a **right rotation** over p, followed by a **left rotation** over g.) to make the target node (20) as root. Note that the parent (30) is rotated first followed by the grandparent (10).

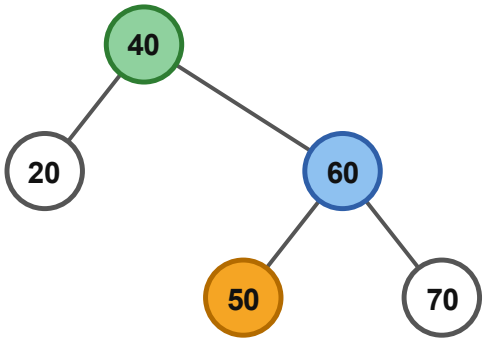
Example 2:

The following diagrams illustrate the searching of the data 50.



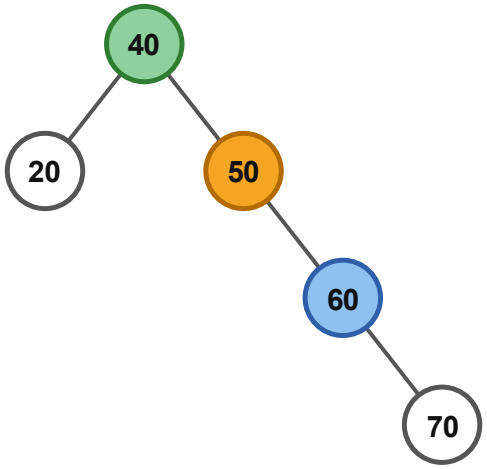
(a) Initial tree

The splay tree before the operation.



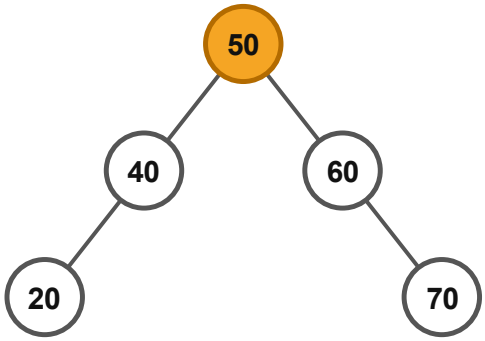
(b) Search path

Search path: 40 -> 60 -> 50. Key 50 is FOUND ; x, p and g are marked.



(c) Step 1: Zag-Zig, rotation 1

Zag-Zig: the path 40 -> 60 -> 50 bends. Rotation 1: Right rotation of 50 over 60.



(d) Step 1: Zag-Zig, rotation 2

Rotation 2: Left rotation of 50 over 40.

Important Point

For Zag-Zig Case, the rotation is performed on the parent first followed by the grandparent. The reverse order does not work in this case, because performing the first rotation on the grandparent would affect the original position of the splaying node.

Insertion in a Splay Tree

Insertion begins similarly to insertion in a Binary Search Tree. The new key is placed at the appropriate leaf position. The newly inserted node is then splayed to the root.

A node is always inserted at the bottom of the tree. Traverse the tree based on the order, insert the new node, and perform splaying to make the new node as root.

Algorithm:

1. Start the traversal from the root node to find the data location.
2. If the data element is smaller than the current node, traverse on the left subtree.
3. Otherwise, traverse on the right subtree if the data element is greater.
4. Insert the node when the traversal reached the bottom.
5. Splay the tree to make the new node as root.

Example 1:

The following diagrams illustrate the insertion of the data 10, 20, 30, 40 and 25.

Insert(10)



Fig. 1: Insert into empty tree

The tree is empty, so 10 simply becomes the root. No splaying needed.

Insert(20)

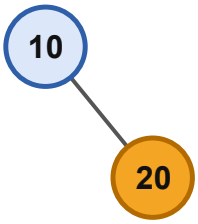


Fig. 1: BST insert

Ordinary BST insertion path: 10. 20 is attached as the right child of 10.

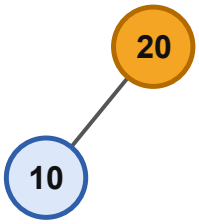


Fig. 2: Splay step 1: Zag

20 is the right child of the root 10: single rotation (left rotate at 10) ->Zag.

Insert(30)

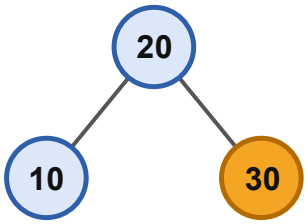


Fig. 1: BST insert

Ordinary BST insertion path: 20. 30 is attached as the right child of 20.

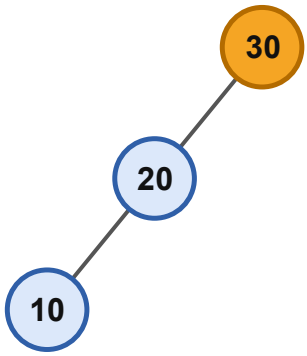


Fig. 2: Splay step 1: Zag

30 is the right child of the root 20: single rotation (left rotate at 20) ->Zag.

Insert(40)

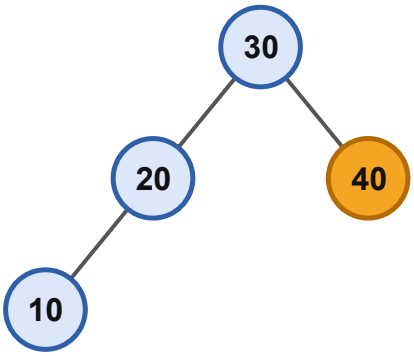


Fig. 1: BST insert

Ordinary BST insertion path: 30. 40 is attached as the right child of 30.

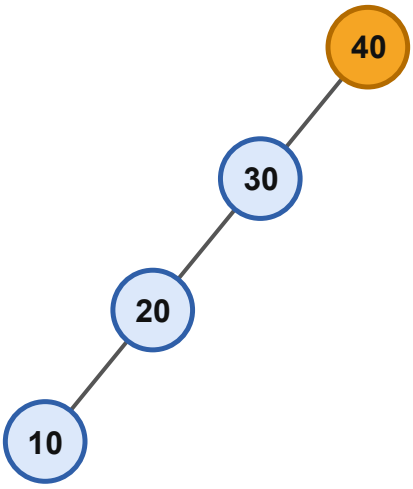


Fig. 2: Splay step 1: Zag

40 is the right child of the root 30: single rotation (left rotate at 30) ->Zag.

Insert(25)

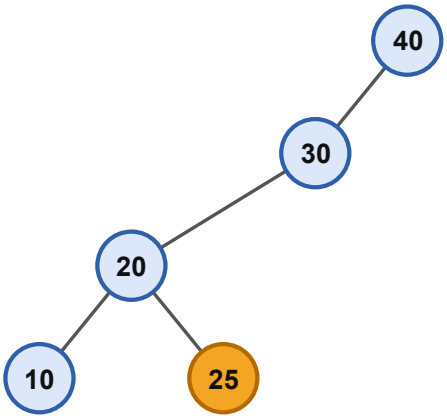


Fig. 1: BST insert

Ordinary BST insertion path: 40 -> 30 -> 20. 25 is attached as the right child of 20.

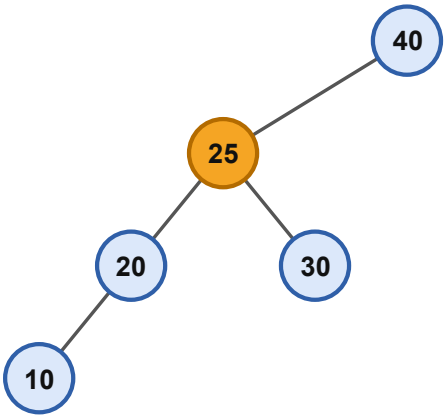


Fig. 2: Splay step 1: Zig-Zag

25 is a right child of 20, but 20 is a left child of 30 (a bend): rotate 25 above 20, then above 30 -> Zig-Zag.

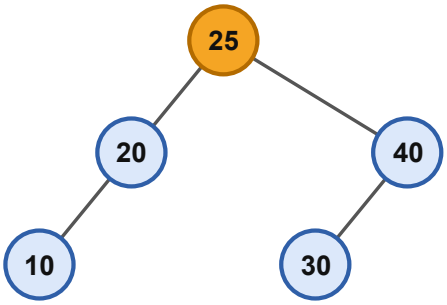


Fig. 3: Splay step 2: Zig

25 is the left child of the root 40: single rotation (right rotate at 40) -> Zig.

Example 2:

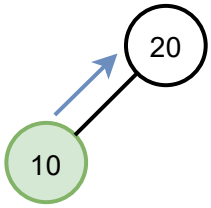
The following diagrams illustrate the insertion of the data 20, 10, 60, 40, 50, and 30.

Insert element 20



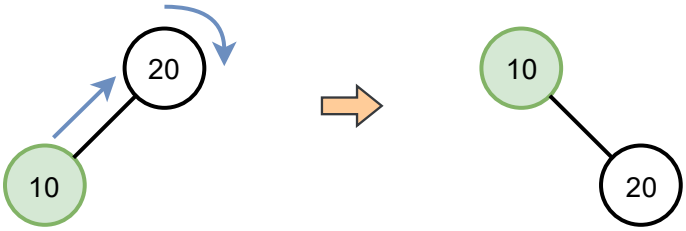
- The initial tree is empty, so insert 20 as the root node.

Insert element 10

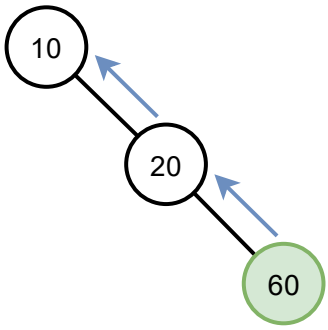


- Create a new node 10 on the left of node (20).
- Subsequently, splay the tree to make node 10 as root.
- The node has only the parent and not the grandparent, so perform a right rotation since the node is on the left side of the root.(Zig Case)

Perform a Right Rotation from the node (20)

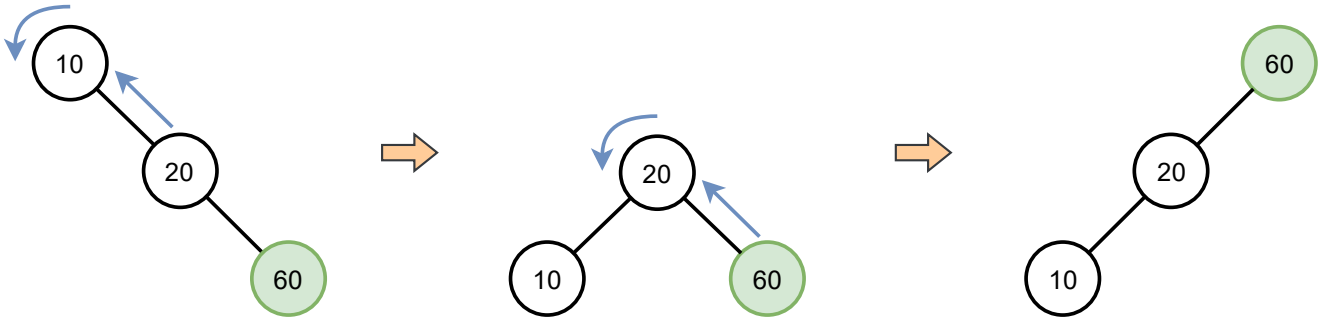


Insert element 60

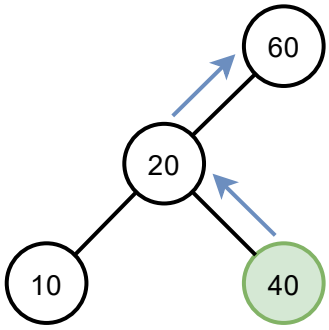


- Create a new node 60 on the right of node (10) and perform splaying.
- The node (60) has the grandparent and is located on the Right-Right side, so perform a Left-Left rotation.(Zag-Zag Case)

Perform a Left-Left Rotation

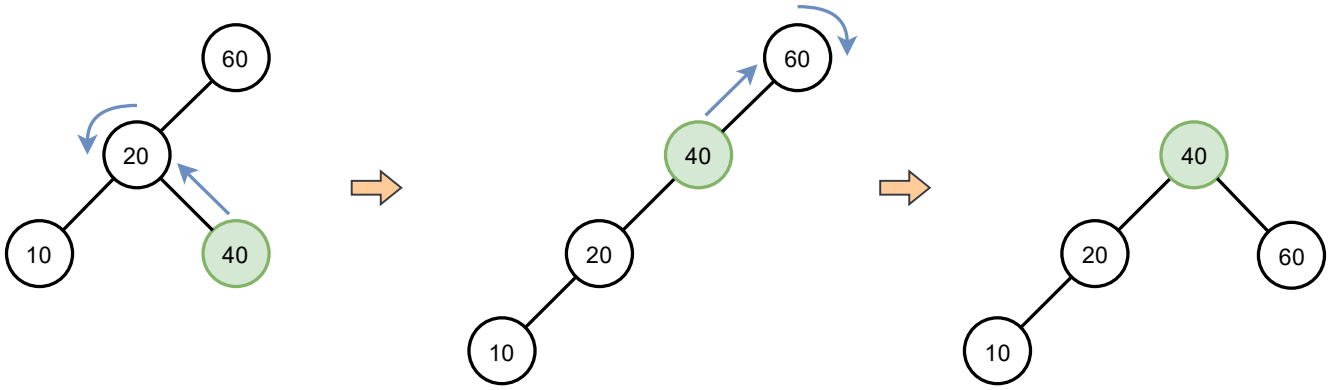


Insert element 40

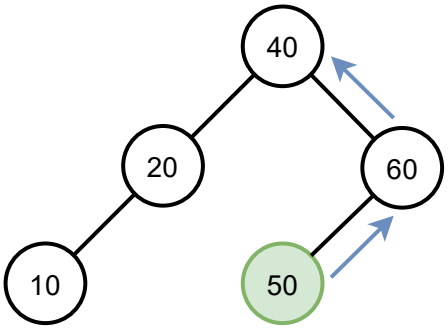


- Create a new node (40) on the right of node (20) and perform splaying.
- The node (40) has the grandparent and is located on the Left-Right side, so perform a Left-Right rotation.(Zig-Zag Case)
- Here we perform the operation on the parent node first.

Perform a Left-Right Rotation

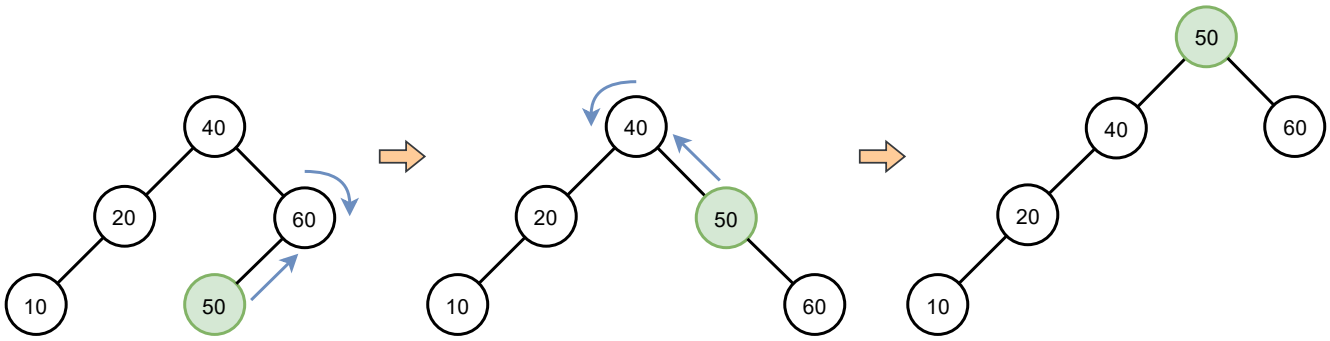


Insert element 50

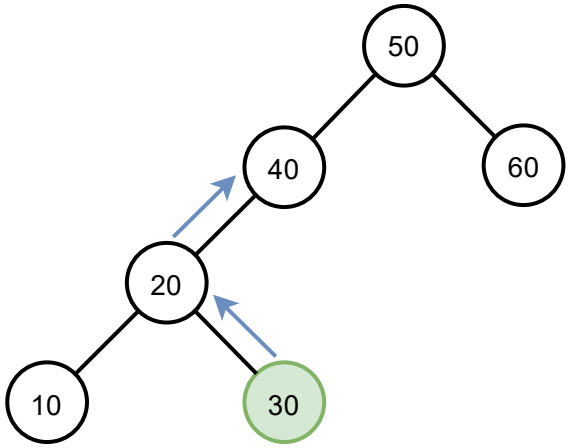


- Create a new node (50) on the left of node (60) and perform splaying.
- The node has the grandparent and is located on the Right-Left side.
- Hence perform a Right-Left rotation to make the new node as root.(Zag-Zig Case)

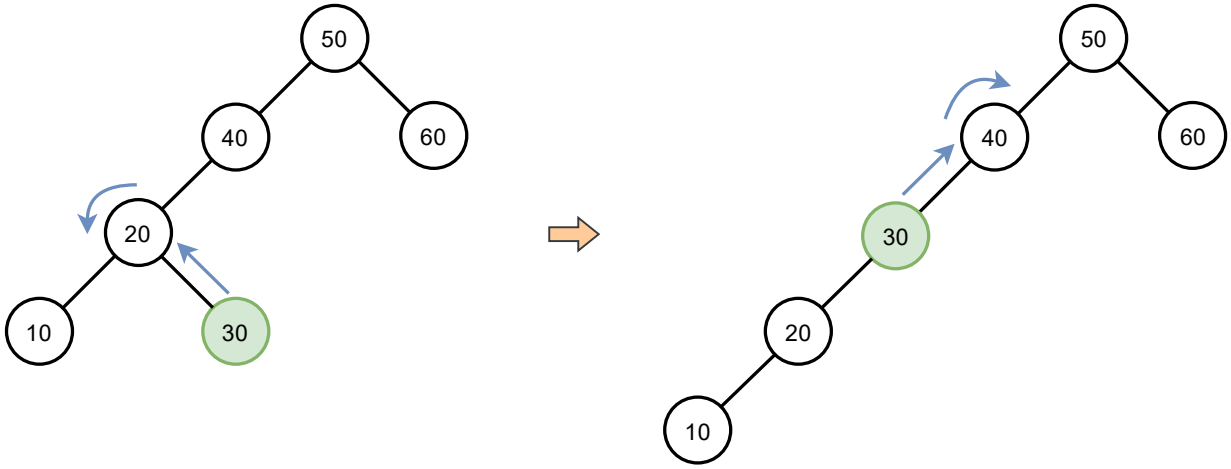
Perform a Right-Left Rotation



Insert element 30

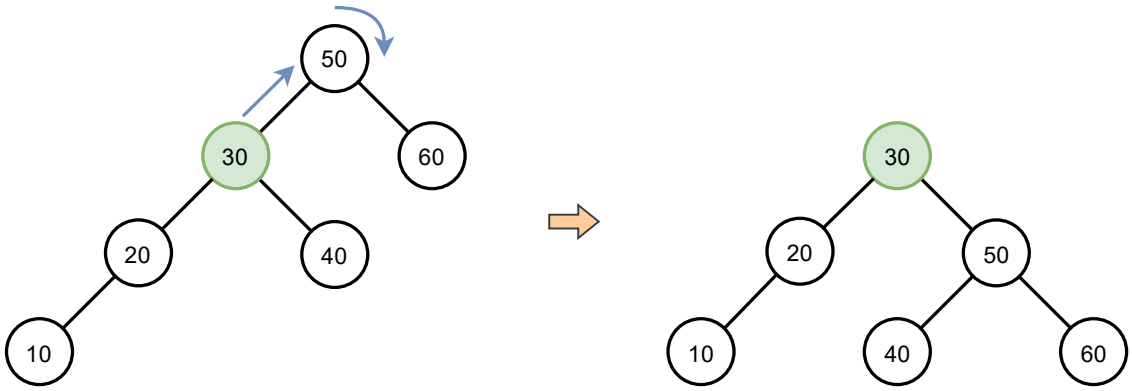


- Create a new node (30) on the right of node (20) and perform splaying.
- The node (30) has the grandparent and is located on the Left-Right side.
- Hence perform a Left-Right rotation to make the new node as root.(Zig-Zag Case)



In The new node (30) is still one level below the root node. Hence repeat the rotation until it becomes the root.

Perform a Right Rotation since the node (30) is on the left of the root.



Searching in a Splay Tree

Searching follows the Binary Search Tree rule. Starting at the root, the search key is compared with the current node. If it is smaller, the search moves left; if larger, it moves right.

The special feature is what happens after the search. If the key is found, the found node is splayed to the root. If the key is not found, a commonly used implementation splays the last accessed node to the root.

Search Algorithm – Concept

1. Start from the root.
2. Compare the target key with the current node.
3. If equal, the node has been found.
4. If target is smaller, move to the left child.
5. If target is larger, move to the right child.
6. When the search finishes, splay the found node or the last accessed node.
7. Return the resulting root.

Example 1:

The following diagrams illustrate the searching of the data 10.

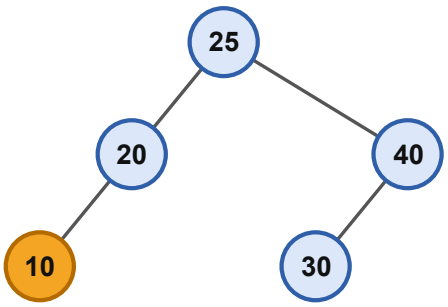


Fig. 1: Search path

Search path: 25 -> 20 -> 10. Key 10 is **FOUND** .

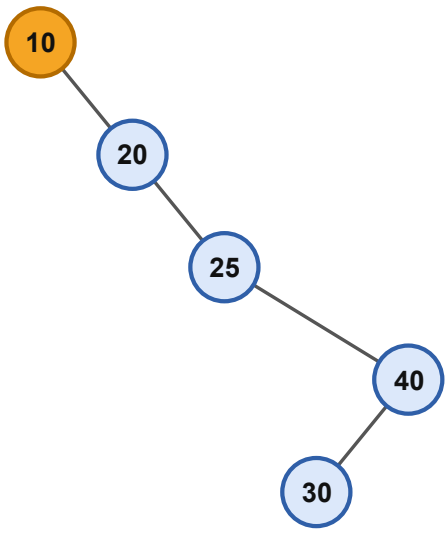


Fig. 2: Splay step 1: Zig-Zig

10 and its parent 20 are both left children: rotate 20 over 25 first, then 10 over 20 -> Zig-Zig.

Unsuccessful Search

If a key is absent, the tree can still be reorganized by splaying the last node encountered. This allows the tree to adapt to unsuccessful access patterns as well.

Example 2:

The following diagrams illustrate the searching of the data 27.

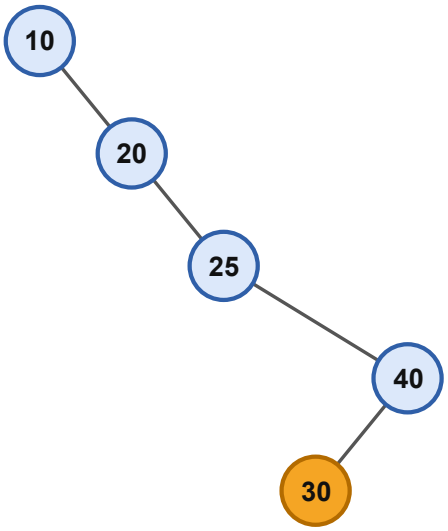


Fig. 1: Search path

Search path: 10 -> 20 -> 25 -> 40 -> 30. Key 27 is **NOT FOUND**; the last node visited is 30.

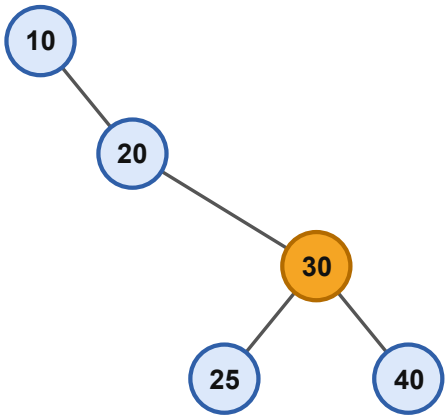


Fig. 2: Splay step 1: Zag-Zig

30 is a left child of 40, but 40 is a right child of 25 (a bend): rotate 30 above 40, then above 25 -> Zag-Zig.

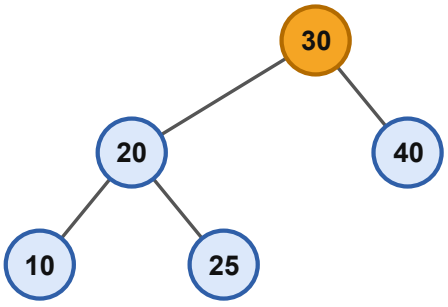


Fig. 3: Splay step 2: Zag-Zag

30 and its parent 20 are both right children: rotate 20 over 10 first, then 30 over 20 -> Zag-Zag.

Deletion in a Splay Tree

Deletion is slightly more involved because the tree must remain a valid BST after the selected node is removed. A common method first splays the target node to the root.

Algorithm :

1. **Search for the key** to be deleted.
2. **Splay the found node to the root.**
3. If the key is not present, **handle the unsuccessful search** according to the implementation.
4. After the target node becomes the root, **separate its left and right subtrees.**
5. If the **left subtree is empty**, make the right subtree the new root.
6. If the **right subtree is empty**, make the left subtree the new root.
7. If **both subtrees are non-empty**, join them using either of the following methods:
    a. **Method 1 – Maximum of the left subtree:** Find the maximum node in the left subtree and splay it within the left subtree so that it becomes its root. Then attach the original right subtree as its right child.
    b. **Method 2 – Minimum of the right subtree:** Find the minimum node in the right subtree and splay it within the right subtree so that it becomes its root. Then attach the original left subtree as its left child.
8. The resulting tree becomes the **new splay tree**.

Example 1: (by method 1)

The following diagrams illustrate the deletion of the data 25.

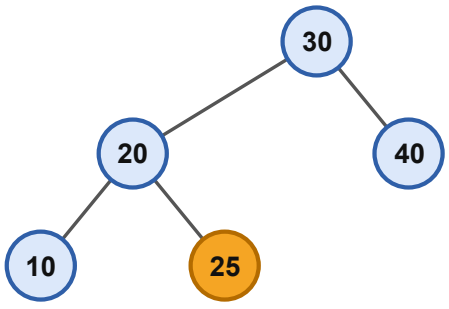


Fig. 1: Search path

Search path: 30 -> 20 -> 25. Key 25 found .

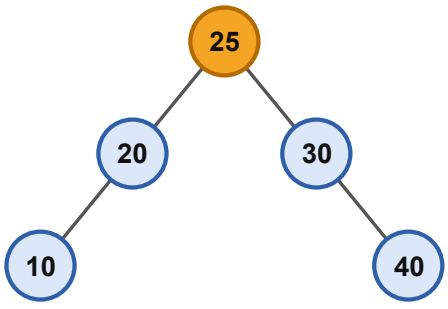


Fig. 2: Splay step 1: Zig-Zag

25 is a right child of 20, but 20 is a left child of 30 (a bend): rotate 25 above 20, then above 30 -> Zig-Zag.

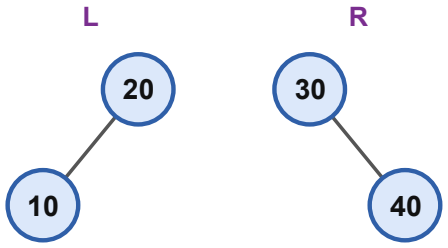


Fig. 3: Remove root 25

Delete the root 25. This splits the tree into a left tree L and a right tree R.

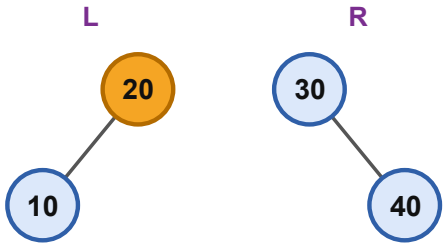


Fig. 4: Find max of L

The largest key in L is 20 .Splay it to the root of L.

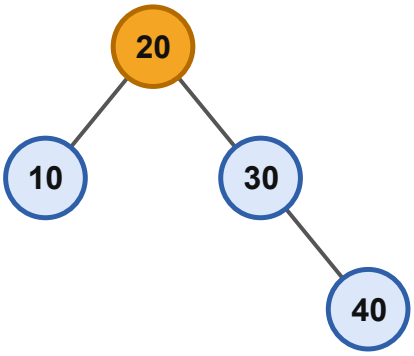


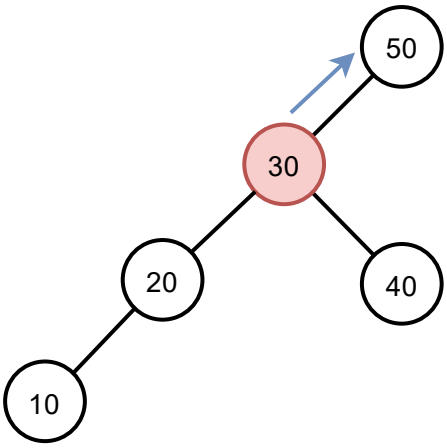
Fig. 5: Join

After splaying, 20 is the root of L and has no right child. Attach R as the right child of 20. Deletion complete.

Example 2:(by Method 2)

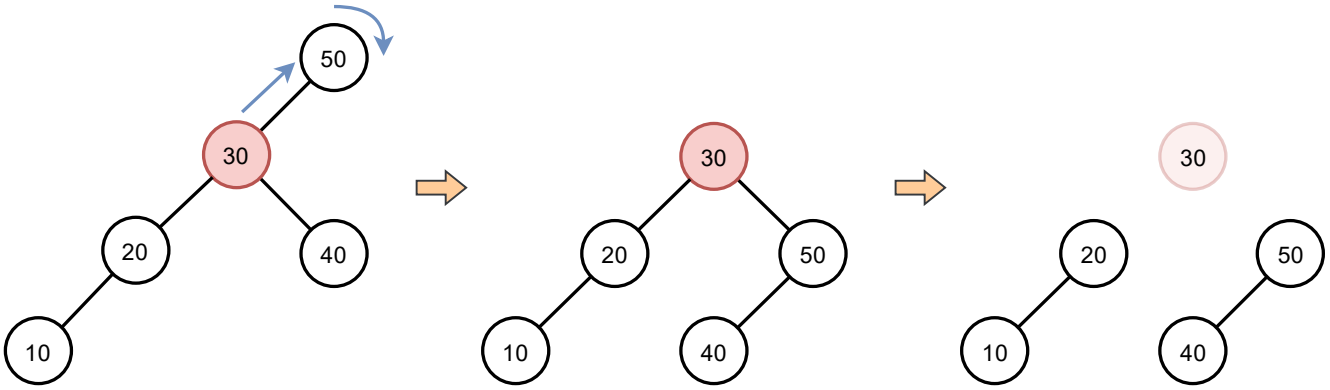
The following diagrams illustrate the deletion of the data 30.

Delete the node 30

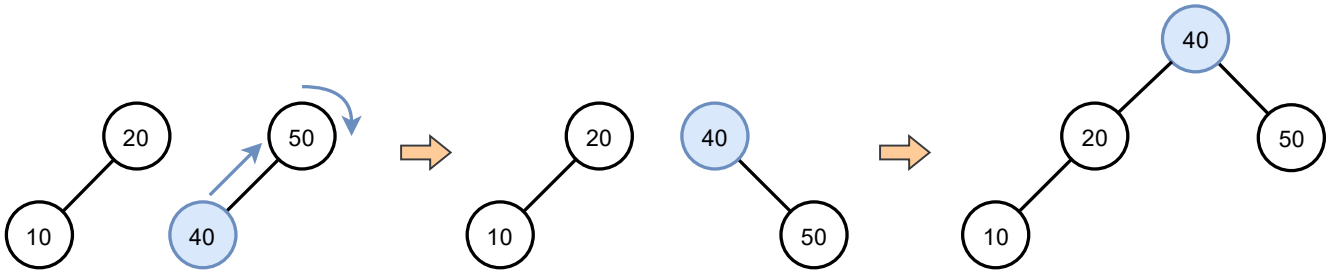


- The node (30) does not have the grandparent and is located on the Left, so perform the Right rotation to make it the root.(Zig Case)
- Find the node (30) and perform Right rotation
- So, the target node (30) becomes the new root

Perform Right Rotation and Delete the node (30)



- Deleting the node (30) separates the left and right subtree. So we need to identify the smaller elements on the right subtree (Inorder-successor), and splay the node to make the Root. Finally, link the left subtree with the new root.



- Merge subtrees after Splay Tree deletion.

Algorithm: Insertion in a Splay Tree

Input: A key K to be inserted
Output: Updated Splay Tree with K as the root

1. Start from the root of the Splay Tree.
2. If the tree is empty, create a new node containing K and make it the root.
3. Otherwise, compare K with the current node.
4. If K is smaller than the current node, move to the left subtree.
5. If K is greater than the current node, move to the right subtree.
6. Continue the traversal until an empty position is reached.
7. Insert the new node at that position following the Binary Search Tree property.
8. Set the newly inserted node as the current node x.
9. Splay x toward the root using the appropriate rotation case:
    - **Zig** – if x is the child of the root.
    - **Zag** – if x is the right child of the root.
    - **Zig-Zig** – if x and its parent are both left children.
    - **Zag-Zag** – if x and its parent are both right children.
    - **Zig-Zag** – if x is the right child of its parent and the parent is the left child of the grandparent.
    - **Zag-Zig** – if x is the left child of its parent and the parent is the right child of the grandparent.
10. Continue the splaying process until x becomes the root.
11. The resulting tree is the updated Splay Tree.

Algorithm: Searching in a Splay Tree

Input: A search key K

Output: Search result and updated Splay Tree

1. Start from the root of the Splay Tree.
2. Set the current node as the root.
3. Compare the search key K with the current node's key.
4. If K is equal to the current node's key:
    - The key is found.
    - Mark the current node as the target node.
    - Splay the target node to the root.
    - Return the resulting root.
5. If K is smaller than the current node's key:
    - Move to the left child.
6. If K is greater than the current node's key:
    - Move to the right child.
7. Continue the process until the key is found or an empty child position is reached.
8. If the key is not found:
    - Mark the last accessed node as the target node.
    - Splay the last accessed node to the root.
9. If the key is found, report **"Key Found"**.
10. If the key is not found, report **"Key Not Found"**.
11. Return the resulting Splay Tree.

Algorithm: Deletion in a Splay Tree

Input: A key K to be deleted

Output: Updated Splay Tree after deletion

1. Start from the root of the Splay Tree.
2. Search for the key K.
3. If K is not found:
    - Splay the last accessed node to the root.
    - Report **"Key Not Found"**.
    - Stop the deletion operation.
4. If K is found, splay the target node K to the root.
5. After splaying, divide the tree into:
    - Left subtree L
    - Right subtree R
6. Remove the root node containing K.
7. If the left subtree L is empty:
    - Make R the new root.
8. Else if the right subtree R is empty:
    - Make L the new root.
9. Otherwise, if both subtrees are non-empty, join them using either of the following methods.

Method 1 – Maximum of the Left Subtree

10. Find the maximum node in the left subtree L.
11. Splay this maximum node within L so that it becomes the root of L.
12. Since the maximum node has no right child, attach the original right subtree R as its right child.
13. Make this joined tree the new Splay Tree.

Method 2 – Minimum of the Right Subtree

14. Find the minimum node in the right subtree R.
15. Splay this minimum node within R so that it becomes the root of R.
16. Since the minimum node has no left child, attach the original left subtree L as its left child.
17. Make this joined tree the new Splay Tree.
18. The resulting tree becomes the updated Splay Tree.

TIME COMPLEXITY OF SPLAY TREE

Introduction

Time complexity describes how the time required by an algorithm changes as the number of elements, represented by **n**, increases.

In a Splay Tree, the time required for an operation depends on the current shape of the tree. Since a Splay Tree is a **self-adjusting Binary Search Tree (BST)**, it changes its structure after an access by moving the accessed node towards the root.

The important point about Splay Trees is that **one operation can sometimes take O(n) time**, but a sequence of operations has an **amortized time complexity of O(log n)**.

Common Time Complexities

Before understanding Splay Tree complexity, it is useful to understand three common notations.

Complexity	Meaning	Simple Example
O(1)	Constant time	Accessing one element directly
O(log n)	Time increases slowly as n increases	Searching in a balanced tree
O(n)	Time increases roughly with the number of elements	Checking all elements one by one

Here, **n** represents the number of nodes in the Splay Tree.

Why Does a Splay Tree Have Different Time Complexities?

A Splay Tree does not always remain balanced like an AVL Tree.

For example, the tree may become **skewed**, where many nodes are arranged mainly on one side.

In such a case, searching for a node may require visiting many nodes.

Therefore, a single operation can take:

O(n) in the worst case.

However, after the node is accessed, the Splay Tree performs **splaying**, which moves that node to the root.

This changes the tree structure and can make future accesses faster.

What is Amortized Time Complexity?

The word **amortized** means considering the total cost of a group of operations instead of looking at only one operation.

In a Splay Tree:

- One operation may take **O(n)**.
- Some other operations may take much less time.
- Splaying changes the tree structure after each access.
- Over a sequence of operations, the **average cost per operation is O(log n)**.

Therefore, the amortized time complexity of the main Splay Tree operations is:

O(log n).

Simple Example

Suppose one operation takes a long time because the required node is far from the root.

After this operation, the node is moved closer to the root or becomes the root itself.

If the same node is accessed again, it can be found much faster.

So, the tree may spend more time on one operation but save time on future operations.

This is the basic idea behind **amortized O(log n)**.

Time Complexity of Splay Tree Operations

The main operations of a Splay Tree are **search, insertion, deletion, and splaying**.

Operation	Best Case	Worst Case for One Operation	Amortized Time
Search	O(1)	O(n)	O(log n)
Insertion	O(1)	O(n)	O(log n)
Deletion	O(1)	O(n)	O(log n)
Splaying	O(1)	O(n)	O(log n)

Splaying Complexity

Splaying is the process of moving an accessed node to the root using rotations.

The required rotations depend on the position of the node.

A node that is already the root requires no rotations:

O(1)

A node that is deep inside a skewed tree may require many rotations:

O(n)

However, considering a sequence of operations, the amortized cost of splaying is:

O(log n)

Space Complexity

The space complexity of a Splay Tree is:

O(n)

This is because each node requires memory to store:

- Data
- Left child pointer
- Right child pointer

If there are **n nodes**, the total memory required grows approximately with **n**.

Therefore:

Space Complexity = O(n)

Worst Case vs Amortized Complexity

Type	Meaning	Splay Tree
Worst Case	Cost of one particularly expensive operation	O(n)
Amortized Case	Average cost over a sequence of operations	O(log n)

A Splay Tree has **O(n) worst-case time for a single operation** and **O(log n) amortized time per operation**.

This is one of the most important characteristics of a Splay Tree.

Time Complexity Conclusion

The time complexity of a Splay Tree is different from that of a strictly balanced tree because a Splay Tree does not maintain a fixed balanced structure.

A single operation can take **O(n)** time in the worst case. However, after each access, the tree adjusts itself by moving the accessed node to the root. Because of this self-adjusting behavior, a sequence of operations has an **amortized time complexity of O(log n)**.

Therefore, Splay Trees are especially useful when some elements are accessed more frequently than others.

Advantages, Disadvantages and Applications

Advantages

- No explicit balance factor, height field or color is required.
- Recently accessed nodes move toward the root.
- Repeated accesses can become very efficient.
- The data structure adapts to changing access patterns.
- The implementation can be relatively compact compared with some balanced trees.

Disadvantages

- A single operation can take O(n) time.
- The tree can become highly skewed temporarily.
- The structure changes after searches, which may be undesirable in some applications.
- Real-time systems may prefer stronger per-operation worst-case guarantees.
- Understanding and debugging rotations can be difficult for beginners.

Applications

Application	Definition
Caching	Splay Trees can be used to manage cached data. Recently accessed data is moved toward the root, which can improve repeated access.
Dynamic Dictionaries	Splay Trees can implement dictionaries where elements are frequently searched, inserted, and deleted. The tree automatically adjusts according to the access pattern.
Symbol Tables	Splay Trees can be used to implement symbol tables that store and retrieve information about identifiers such as variables and functions.
File and Directory Management	Splay Trees can organize searchable file and directory information. Frequently accessed entries are moved closer to the root.
Non-Uniform Access Data	Splay Trees are suitable when some elements are accessed much more frequently than others because the tree automatically adapts to the changing access pattern.

Comparison of BST, Splay Tree and AVL

Feature	BST	Splay Tree	AVL Tree
Balancing method	None	Self-adjusting	Strict height balance
Extra metadata	Usually none	None	Height/balance factor
Single search worst case	O(n)	O(n)	O(log n)
Adapts to access pattern	No	Yes	No
Rotations	Not required normally	Frequent during splay	Used for balancing
Implementation idea	Simple BST	BST + splay	BST + height balance

Conclusion

- A Splay Tree is a **self-adjusting Binary Search Tree (BST)** that automatically reorganizes itself after an access by moving the accessed node to the root. This self-adjusting behaviour allows the tree to adapt to changing access patterns and makes frequently accessed elements easier to access again.
- Splaying is performed using rotations such as **Zig, Zag, Zig-Zig, Zag-Zag, Zig-Zag and Zag-Zig**. These rotations preserve the BST ordering property while changing the structure of the tree.
- Unlike AVL Trees, a Splay Tree does not store extra height or balance information.
- Therefore, Splay Trees are especially useful when the same elements are accessed repeatedly or when the access pattern has **locality**. They provide a simple and adaptive alternative to strictly balanced trees.

References

The following resources were referred to for understanding, researching and preparing this project on **Splay Trees**.

1. GeeksforGeeks

https://www.geeksforgeeks.org/dsa/introduction-to-splay-tree-data-structure/

2. Simplerize

https://simplerize.com/data-structures/splay-tree

3. AI Tool – ChatGPT (OpenAI)

https://chatgpt.com/

Note: These resources were used for educational reference and concept understanding. The project content was compiled, reviewed and organised collaboratively by the team.